Data Cleaning Using Apache Spark: Duplicate Data

Explore techniques to clean data by removing or aggregating duplicate records using Apache Spark. Understand how to detect identical and near-identical duplicates, use Spark SQL for querying, and apply methods like filtering and aggregation to maintain data integrity in ETL processes.

Another aspect of data cleaning is removing duplicate data. The goal is to ensure that each record in the data is unique and that no duplicate records exist. It is important because duplicates can lead to inaccurate results, cause errors when loading the duplicate data into an existing schema, and negatively impact data analysis and reporting.

Duplicates can happen for many reasons: human error, bugs, and errors when joining/merging data from different sources.

Handling duplicate data

Similar to handling missing data, we should try to understand whether a duplicate value needs to be removed. It requires some business context, and our actions will depend on the desired outcome.

When we find duplicate records of data, we can remove them, aggregate them into single records, we can mark them with an additional column, or just keep them as they are. Generally, there are two main types of duplicate data:

Ask

1. Identical duplicates

2. Near-identical duplicates

Let's see an example of handling identical and near-identical duplicate records. For this demonstration, we'll use a CSV file containing customer data. This table is stored in an OLTP relational database. Therefore, all rows must be unique and identified by the primary keys.

The primary keys in this case are:

1. `customer_id`
2. `first_name`, `last_name`, and `address`

A sample of the data looks like this:

main.csv Python 3.10.4

Sample customer data

Identical duplicates

Dealing with identical duplicates is the easiest. Unless there's a particular reason why a record is duplicated, we should simply drop all identical records except for one. First, we should check if there are identical duplicates in this dataset.

We'll create a Spark session and load the CSV file containing the customer's data into a Spark DataFrame.

main.py Python 3.10.4

```
from pyspark.sql import SparkSession

# Create a spark session
print('\nCreating a Spark sessions')
spark = SparkSession.builder.appName('DuplicateData').getOrCreate()

# Load CSV file
print('\nLoading the data')
customer_df = spark.read.option('header', 'true').option('inferSchema', '
.csv('/usercode/tmp/customers.csv')

# Print DataFrame schema
print('\nResults:')
```

Run

Loading customers.csv

- **Line 5:** We create a spark session. A Spark session (`SparkSession`) is the entry point to using Pyspark. It creates DataFrames and provides access to Spark's configuration options.
- **Line 9:** We load the CSV file. We can let Spark infer the format of the CSV file. To do that, we should hint that the file has headers.
- **Line 14:** We print the DataFrame.

Then, we can use Spark's SQL API to perform SQL queries on the DataFrame. Let's use SQL to search for identical duplicates.

main.py Python 3.10.4

Ask

```
# Creating a temporary view of the DataFrame
print('\nCreating a temporary view of the DataFrame')
customer_df.createOrReplaceTempView('customers')
```

```
# SQL Query - select all duplicate records
query = '''
select
    first_name,
    last_name,
    address,
    height,
    weight,
```

Run

Collecting all identical duplicates

- **Line 3:** We create a temporary view of the DataFrame called **customers**. This allows us to use Spark's SQL API to query the DataFrame.
- **Lines 6–26:** We select all identical duplicates in the DataFrame.
- **Line 31:** We apply the SQL query to the temporary view `customers`.
- **Line 35:** We print the duplicate rows.

Looks like there are two identical customers named `Daenerys Targaryen`. We should discard one of them. We can use the window function `row_number()` to assist with this. The following query will assign a row number equal to 1 for each distinct record. Duplicate records will incrementally receive a higher row number for each duplicate. It will look like this:

main.py Python 3.10.4

```
# SQL Query - Assigning row numbers
query = '''
select
    c.*,
    row_number() over (partition by
                                    customer_id,
                                    first_name,
                                    last_name,
                                    address,
                                    height,
                                    weight
                            order by customer_id) as row_number
```

Ask

Run

Marking the duplicate records by assigning incremental row numbers

- **Lines 3–14:** We assign incrementally increasing row numbers to duplicate rows.
- **Line 19:** We apply SQL query on `customers`.
- **Line 23:** We print the results.

As we can see, the only record with a `row_number` higher than 1 is a duplicate record that should be removed. We can filter it using the following query:

main.py Python 3.10.4

```
# SQL Query - Remove identical duplicates
query = '''
select
    customer_id,
    first_name,
    last_name,
    address,
    height,
    weight
from
(
    select
```

Run

Discarding one of the duplicate records

- **Lines 3–26:** We remove records with a `row_number` value higher than 2.
- **Line 31:** We apply SQL and remove duplicate records.
- **Line 35:** We print the updated DataFrame.

Ask

Near-identical duplicates

Handling near-identical duplicates can be harder than identical ones because it's difficult to be sure if the records are duplicated by mistake or on purpose. For example, records in a customer table can have the same first and last name. Should we drop these records?

Of course not. Having two customers with the same first and last name is common.

What about customers with the same first name, last name, and address?

This is where it can get tricky. Answering questions like this might require us to talk to other company employees to understand why this might have happened. They also require an understanding of the systems we operate in.

For example, having the same first name, last name, and address violates the primary key constraints of the customer's table from which the data came from. Therefore, we can be sure this is a mistake, and we need to discard one of these near-identical records.

First, let's collect records with the same first name, last name, and address.

main.py Python 3.10.4

```
# Creating a temporary view of the DataFrame
print('\nCreating a temporary view of the DataFrame')
customer_df.createOrReplaceTempView('customers')

# SQL Query - select all near-duplicate records
query = '''
select
    first_name,
    last_name,
    address,
    count(*) as duplicate_rows
from customers
group by
```

Ask

Run

Collecting all near-identical records

Looks like there are two customers named `Jon Snow` located at `The Wall`. We need to decide what to do with these records. Let's take a closer look.

main.py Python 3.10.4

```
# SQL Query - select all near-duplicate records
query = '''
select *
from customers
where
    first_name = 'Jon' and
    last_name = 'Snow' and
    address = 'The Wall';
'''

# Applying SQL query
near_duplicate_rows = spark.sql(query)
```

Run

Querying all customers named Jon Snow from The Wall

As we can see, these two records have different heights and weights. Without any further knowledge of this particular case, deciding which one is the correct record might be impossible. At this point, we might need to consult other factors in the organization or contact the client. Some steps we take are:

1. Aggregate these two records into one
2. Mark one as duplicated
3. Or, keep them as they are

For this demonstration, we'll aggregate the two records into one record containing their average height and weight. First, we'll collect the average height and of the two records and discard one.

Ask

main.py Python 3.10.4

```
from pyspark.sql.functions import when, avg

# Creating a temporary view of the DataFrame
print('\nCreating a temporary view of the DataFrame')
customer_df.createOrReplaceTempView('customers')

print('\nCalculating average height and weight of customers with first_na
average_height = round(customer_df.filter(customer_df.first_name == 'Jon'
average_weight = round(customer_df.filter(customer_df.first_name == 'Jon'
print(f'Average height: {average_height:.0f}')
print(f'Average weight: {average_weight}')

# SQL Query - Remove one of the near identical records
```

Run

Collecting the average height and weight and discarding one of the two near-identical records

- **Line 8:** We calculate the average height of all customers named `Jon`.
- **Line 9:** We calculate the average weight of all customers named `Jon`.
- **Lines 15–35:** We remove one of the customers named `Jon Snow`.
- **Line 40:** We apply SQL query on `customers`.

Next, we'll update the remaining record with the average `height` and `weight` that we just collected.

main.py Python 3.10.4

```
# Updating the height and weight values to the average
print("\nReplacing Jon's height and weight with the average height and we
customer_df = customer_df.withColumn("height", when(customer_df.first_nam
.otherwise(customer_df.height))

customer_df = customer_df.withColumn("weight", when(customer_df.first_nam
.otherwise(customer_df.weight))

# Creating a temporary view of the DataFrame
print('\nCreating a temporary view of the DataFrame')
customer_df.createOrReplaceTempView('customers')
```

Ask

```
# SQL Query - select all near duplicate records
```

Run

Updating the height and weight of Jon Snow to the average height and weight

- **Line 3:** We update the remaining customer named `Jon Snow`'s height to the average height.
- **Line 6:** We update the remaining customer named `Jon Snow`'s weight to the average weight.
- **Line 11:** We create an updated view of `customers`.
- **Lines 15–20:** We verifying the new `height` and `weight` of `Jon Snow`.
- **Line 24:** We apply SQL query on `customers`.
- **Line 28:** We print the results.

Ask